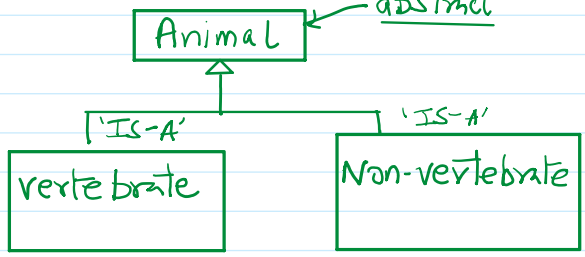
# Abstraction in Java

08 January 2026 20:47

**Abstract class in Java**

In Java programming, Abstract classes are used to define the structure and behavior of classes within an inheritance hierarchy. They act as a blueprint for other classes where some methods may be declared without implementation.




**What is Abstraction in Java?**

Abstraction is the process of hiding **implementation Details** And **showing only the required functionality** to the user. In other words, it displays only the essential features while hiding internal details.

For example, when sending an SMS we type the message and send it without knowing how the message is processed or delivered internally.

Abstraction allows you to focus on **what an object does** rather than **how it does it.**

**There are two ways to achieve abstraction in Java:**

- **Using abstract class. (Can provide 0% to 100% abstraction)**
- **Using Interface(Provides one 100% abstraction)**

**What is Abstract class in Java?**

A class declared using the **abstract** keyword is known as **abstract class** in Java. It can contain both abstract methods(Method without a body) And non-abstract method (methods with a body).

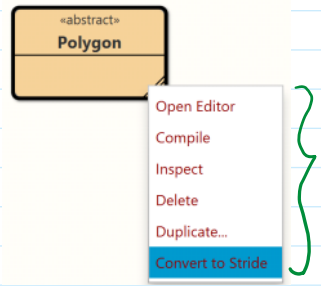
An abstract class provide partial abstraction. It cannot be created on its own and is used only as a parent class. It defines a common structure and shared behavior. While child class provides their own implementations. Abstract method do not have a body and must be implemented by the subclass.

**Rules for Java Abstract Class**

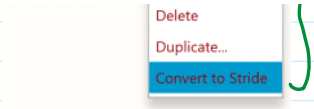
- **An abstract class must be declared with an abstract keyword.**
- **It can have abstract and non-abstract method.**
- **It cannot be instantiated.**
- **It can have constructors and static methods also.**
- **It can have final methods which will force the subclass not to change the body of the method.**

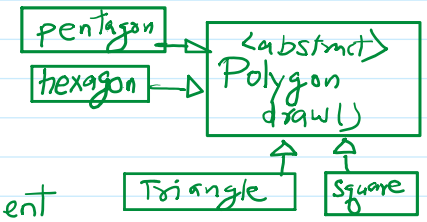
**Creating/Declaring an Abstract Class**

An abstract class is created using the **abstract** keyword. It is used as a parent class and cannot be instantiated directly.




There is no option to create an object of abstract class Polygon.

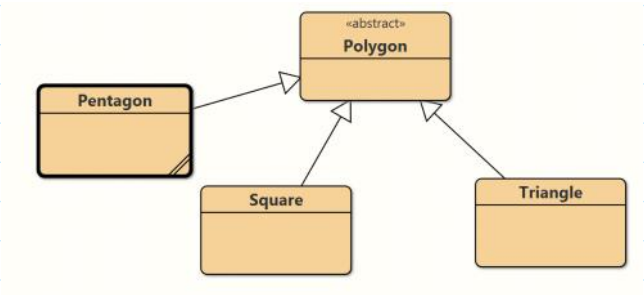


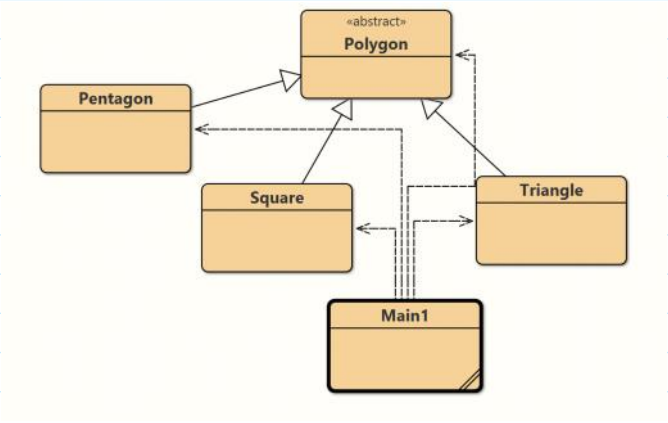

Java compiler is forcing us to implement abstract method draw().

```
10 public class Square extends Polygon
11 {
12     |
13 }
```

AbstractionInJava.Square is not abstract and does not override abstract method draw() in AbstractionInJava.Polygon




**Abstraction of Classes in Java**




```
public abstract class Polygon
{
    public abstract void draw(); //prototype of the method
    public void display(){
        System.out.print("\nThis is a polygon.");
    }
}

public class Pentagon extends Polygon
{
    @Override
    public void draw(){
        System.out.print("\nDrawing pentagon...");
    }
}
```

abstract method (no method body).

Keyword.

non-abstract method.

```
public class Square extends Polygon
{
    @Override
    public void draw(){
        System.out.print("\nDrawing square...");
    }
}

public class Triangle extends Polygon
{
    @Override
    public void draw(){
        System.out.print("\nDrawing Triangle...");
    }
}

public class Main1
{
    public static void main(String[] args){
        System.out.print("\f");
        Triangle t1 = new Triangle();
        Square sq1 = new Square();
        Pentagon p1 = new Pentagon();

        Polygon p = t1; //assigning the reference of Triangle object t1 in p
        p.draw(); //it will call Triangle class draw() method

        p = sq1; //assigning the reference of Square object sq1 in p
        p.draw(); //it will call Square class draw() method

        p = p1; //assigning the reference of Pentagon object p1 in p
        p.draw(); //it will call Pentagon class draw() method

        Polygon pa[] = {t1, sq1, p1};
        System.out.print("\n=======================");
        for(Polygon pq : pa){
            pq.draw(); //depending upon the reference it will call the draw method
            pq.display(); //calling non-abstract method
        }
    }
}
```

**Output:**

```
BlueJ: Terminal Window - AaditClass7
Options

Drawing Triangle...
Drawing square...
Drawing pentagon...
=======================
Drawing Triangle...
This is a polygon.
Drawing square...
This is a polygon.
Drawing pentagon...
This is a polygon.
```

output of non-abstract method of abstract class Polygon.

**Interface in Java**
An interface in Java is used to define a common set of methods that a class must implement. An interface helps in achieving abstraction and supports multiple inheritance.

**What is Interface in Java?**
An interface is a blueprint of a class that contains **static constants** and **abstract methods**. Interfaces are used to achieve abstraction. An interface contains only abstract methods (methods without a body) and variables. It cannot be instantiated similar to an abstract class and represents a **IS-A** relationship.

Since Java 8, interfaces can include **default** and **static methods.**
Since Java 9, interfaces can also contain **private methods.**

**Why Use an Interface in Java?**
An interface is used in Java for the following reasons:
- **To achieve abstraction by defining method signatures without implementation.**
- **To support multiple inheritance,** as a class can implement multiple interfaces.
- **To achieve loose coupling:** Making the code more flexible and easier to maintain.

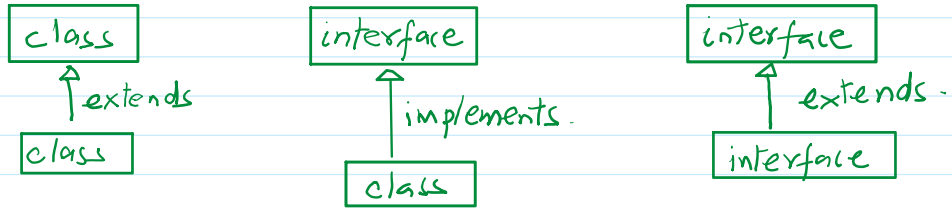
**Declaring an interface**
An interface is declared using the interface keyword. It provides a complete abstraction, which means all methods are abstract by default and all fields are public, static and final. A class that implements an interface must provide implementations for all the methods declared in the interface.

**Syntax of Interface Declaration**

```
interface <interface_name> {
  //declare constant fields
  //declare abstract methods
}
```

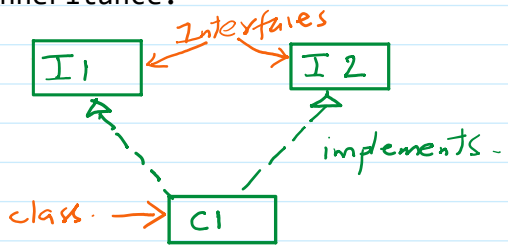
**Implicit modifiers are added by the Java compiler.**
In a Java interface, the compiler automatically adds a certain modifier to its members. All methods are implicitly declared as public and abstract, even if these keywords are not explicitly written. Similarly, all fields declared in an interface are implicitly public, static and final.
In other words, interface methods are public and abstract by default. While interface fields are public, static and final by default.

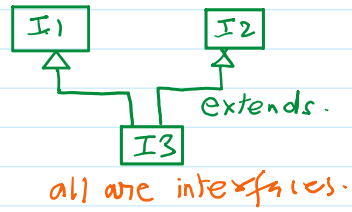
**Relationship between classes and interfaces**
As shown in the following figure, a class extends another class, an interface extends another interface, and a class implements an interface. This relationship defines how classes and interfaces interact and support inheritance and abstraction in Java.

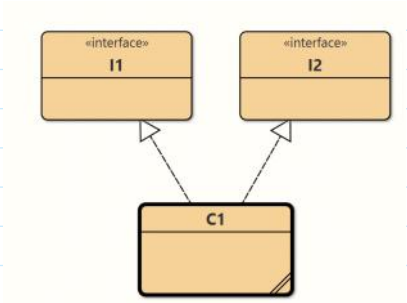


**Multiple inheritance**

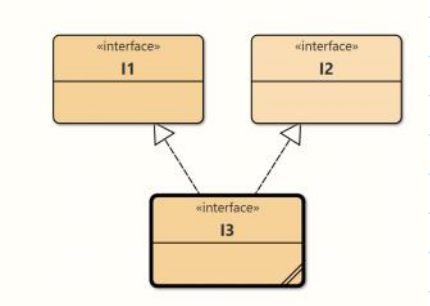
In Java, a class can implement multiple interfaces, so it is called multiple inheritance. Another instance is an interface can extend multiple interfaces, so it is also called multiple inheritance.



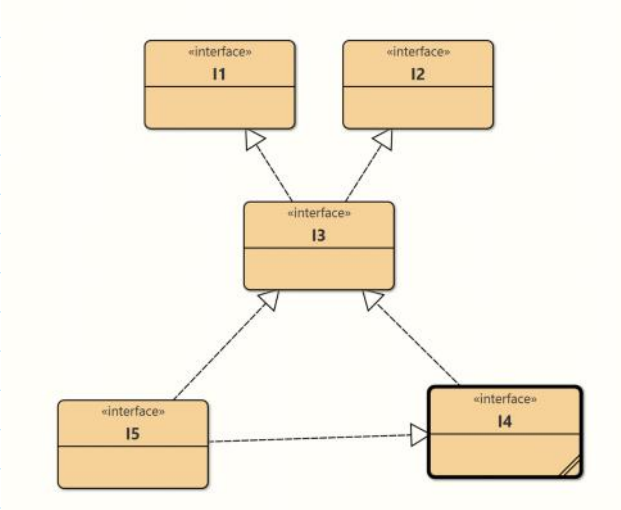




Multiple Inheritance using interfaces and class.



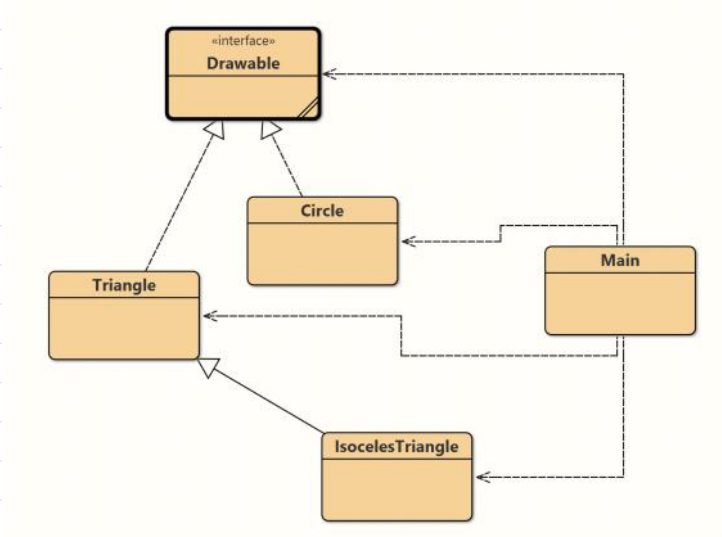

Multiple inheritance using interfaces only.

**Hybrid inheritance**




**Hybrid inheritance and Multiple Inheritance is only possible using interfaces because interfaces are skeleton of code, not actual code.**

**Example:**




```
public interface Drawable
{
    void draw();
}

public class Circle implements Drawable
{
    @Override
    public void draw(){
        System.out.print("\nDrawing circle: ");
    }
}


public class Triangle implements Drawable
{
    @Override
    public void draw(){
        System.out.print("\nDrawing Triangle...");
    }
}

public class IsocelesTriangle extends Triangle
{
    @Override
    public void draw(){
        System.out.print("\nCalling super class method from IsocelesTriangle"+
            "class draw()...");
        super.draw();
        System.out.print("\nDrawing Isoceles Triangle...");
    }
}

import java.util.*;
public class Main
{
    public static void main(String[] args){
        ArrayList<Drawable> d = new ArrayList<>();
```

```
        System.out.print("\f");
        d.add(new Circle());
        d.add(new Triangle());
        d.add(new IsocelesTriangle());

        for(Drawable dr : d){
            dr.draw();
        }
    }
}
```

**Output:**

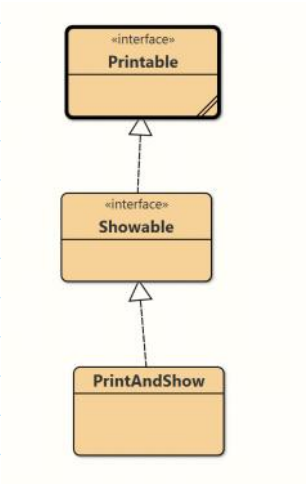
```
BlueJ: Terminal Window - AaditClass7
Options

Drawing circle:
Drawing Triangle...
Calling super class method from IsocelesTriangleclass draw()...
Drawing Triangle...
Drawing Isoceles Triangle...
```

**Inheritance of Interfaces**

A class implements an interface, whereas an interface can extend another interface. When one interface extends another, it inherits all the abstract methods of the parent interface.

A class that implements the child interface must provide implementation for all methods declared in both the parent and child interfaces.

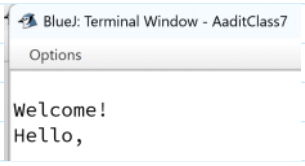
Example:




```
public interface Printable
{
    void print();
}

public interface Showable extends Printable
{
    void show();
}
```

```java
public class PrintAndShow implements Showable
{
    @Override
    public void show(){
        System.out.print("\nHello, ");
    }

    @Override
    public void print(){
        System.out.print("\nWelcome!");
    }

    public static void main(String[] args){
        System.out.print("\f");
        PrintAndShow ps = new PrintAndShow();
        ps.print();
        ps.show();
    }
}
```

**Output:**




**Homework**

**Question 1 – Abstract Class Implementation**
Create an abstract class named Vehicle with the following:
Requirements:
- One abstract method: startEngine()
- One concrete method: fuelType() that prints *"Vehicle uses fuel."*

Create two subclasses:
1. Car
2. Bike

Tasks:
- Override startEngine() in both subclasses.
- In Car, print *"Car engine started using key."*
- In Bike, print *"Bike engine started using self-start."*

Main Class Tasks:
- Create objects of Car and Bike
- Use a Vehicle reference to call methods
- Demonstrate runtime polymorphism
- Display outputs properly

**Question 2 – Shape Drawing Using Abstract Class**
Create an abstract class Shape:
Methods:
- Abstract method draw()
- Concrete method display() that prints *"This is a shape."*

Create subclasses:

- Circle
- Rectangle
- Triangle

Tasks:
- Implement draw() method in each subclass.
- Store objects in an array of Shape references.
- Use a loop to call draw() and display() for each object.

Expected Learning:
- Abstraction
- Method overriding
- Polymorphism

Question 3 – Interface Based Payment System

Create an interface Payment:

Method:
- payAmount(double amt)

Create classes:
- CreditCardPayment
- UPIPayment

Tasks:
- Implement the interface in both classes.
- In CreditCardPayment, print *"Paid Rs. using Credit Card"*
- In UPIPayment, print *"Paid Rs. using UPI"*

Main Class Tasks:
- Store objects in an ArrayList<Payment>
- Call payAmount() using loop

Question 4 – Multiple Inheritance Using Interfaces

Create two interfaces:

Interface 1: Printable
- Method: print()

Interface 2: Scannable
- Method: scan()

Create class MultiFunctionPrinter that implements both.

Tasks:
- Implement both methods.
- Print suitable messages.
- Create object and call both methods.

Objective:
- Understand multiple inheritance using interfaces.

Question 5 – Interface Inheritance Program

Create two interfaces:

Interface 1: Readable
- Method: read()

Interface 2: Writable extends Readable
- Method: write()

Create class Notebook that implements Writable.

Tasks:
- Implement both methods.
- Print meaningful messages.
- Create main method to demonstrate.

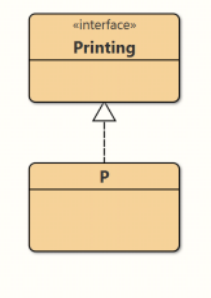
**Interfaces and Polymorphism**

One of the main benefits of interfaces is that they support polymorphism. An interface reference can point to any object of the class that implements it. This allow the program to decide at runtime which method implementation to use.

```
package TestingPrint;
import AbstractionInJava.InterfacesInJava.*;
import java.util.*;

/**
 * Write a description of class PrintingPattern here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class PrintingPattern
{
    /**
     * 1
     * 1 2
     * 1 2 3
     * 1 2 3 4
     * 1 2 3 4 5
     */

    public static void main(String[] args){
        P p = new P();
        p.print("\f");
        int i, n;
        Scanner sc = new Scanner(System.in);
        p.print("Enter number of lines: " );
        n = sc.nextInt();
        for(i = 1; i <= n; i++){
            for(int j = 1; j <= i; j++){
                p.print(j+" ");
            }
            p.println();
        }
    }
}
```

```
package AbstractionInJava.InterfacesInJava;


/**
 * Write a description of interface Print here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public interface Printing
{
    public void print(String str);
    public void println(String str);
```

```
    public void println();
}

package AbstractionInJava.InterfacesInJava;


/**
 * Write a description of class P here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class P implements Printing
{
    @Override
    public void println(String str){
        System.out.println(str);
    }

    @Override
    public void print(String str){
        System.out.print(str);
    }

    @Override
    public void println(){
        System.out.println();
    }
}
```




**Static methods are possible in interface after the version-1.9**

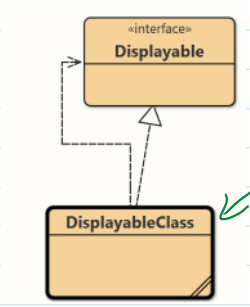
```
package AbstractionInJava.InterfacesInJava;


/**
 * Write a description of interface Displayable here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public interface Displayable
{
    public static void print(String str){
        System.out.print(str);
    }

    public static void println(String str){
        System.out.println(str);
    }

    public static void println(){
        System.out.println();
    }
```

```
}
```

Test code:

```
package TestingPrint;
import AbstractionInJava.InterfacesInJava.*;
import java.util.*;

/**
 * Write a description of class DisplayingPattern here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class DisplayingPattern implements Displayable
{
    /**
     * 1
     * 1 2
     * 1 2 3
     * 1 2 3 4
     * 1 2 3 4 5
     */

    public static void main(String[] args){
        Displayable.print("\f");
        int i, n;
        Scanner sc = new Scanner(System.in);
        Displayable.print("Enter number of lines: " );
        n = sc.nextInt();
        for(i = 1; i <= n; i++){
            for(int j = 1; j <= i; j++){
                Displayable.print(j+" ");
            }
            Displayable.println();
        }
    }
}
```

**Interface supports default methods from version 1.8**




Factory class which contains a factory method, which is used to create instances of Abstract class or interface.

```
package AbstractionInJava.InterfacesInJava;


/**
 * Write a description of interface Displayable here.
```

```
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public interface Displayable
{
    static void print(String str){
        System.out.print(str);
    }

    static void println(String str){
        System.out.println(str);
    }

    static void println(){
        System.out.println();
    }

    default void msg(){
        System.out.print("\nThis displayable interface is designed during Aadit's
class.");
    }
}

package AbstractionInJava.InterfacesInJava;


/**
 * Write a description of class DisplayableClass here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
//factory class
public class DisplayableClass implements Displayable
{
    //factory method use to create instances of Abstract types or Interfaces
    public static Displayable createObject(){
        return new DisplayableClass();
    }
}

package TestingPrint;
import AbstractionInJava.InterfacesInJava.*;
import java.util.*;

/**
 * Write a description of class DisplayingPattern here.
 *
 * @author (your name)
 * @version (a version number or a date)
 */
public class DisplayingPattern implements Displayable
{
    /**
```

```
     * 1
     * 1 2
     * 1 2 3
     * 1 2 3 4
     * 1 2 3 4 5
     */

    public static void main(String[] args){
        Displayable.print("\f");
        int i, n;
        Scanner sc = new Scanner(System.in);
        Displayable.print("Enter number of lines: " );
        n = sc.nextInt();
        for(i = 1; i <= n; i++){
            for(int j = 1; j <= i; j++){
                Displayable.print(j+" ");
            }
            Displayable.println();
        }

        Displayable d = DisplayableClass.createObject();
        d.msg();
    }
}
```

← calling default method of interface.

needs object reference

**Interface Vs. Abstract Class**

An abstract class can have both abstract and concrete methods or instance methods, constructors, and different types of variables, but it does not support multiple inheritance. An interface can only declare abstract methods (with default and static methods allowed since Java Version 1.8), has no constructors, and support multiple inheritance.

Scanner sc = new Scanner(System.in);

sc.nextInt()

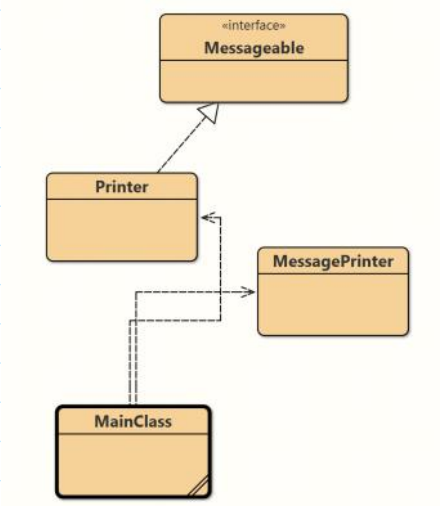
object

concrete method or instance method.

**Nested Interface**

An interface can be declared inside another interface. Such interface is called nested interface. A class can implement. The outer interface. The nested interface or both depending on the requirement.

Example:




```
public interface Messageable
{
    void msg();

    //Nested interface
    interface PrintableMessage{
        void msg();
        void printMsg();
    }
}

public class Printer implements Messageable
{
    @Override
    public void msg(){
        System.out.print("\nThis is outer interface method.\n");
    }
}

public class MessagePrinter implements Messageable.PrintableMessage
{
    //nested or inner interface methods
    @Override
    public void printMsg(){
        System.out.print("\nThis printMsg() is nested interface method.\n");
    }

    @Override
    public void msg(){
        System.out.print("\nThis msg() is nested interface method.\n");
    }
}

public class MainClass
{
    public static void main(String[] args){
        System.out.print("\f");
        Printer p = new Printer();
```

```
        p.msg();

        MessagePrinter mp = new MessagePrinter();
        mp.msg();
        mp.printMsg();
    }
}
```

Output:

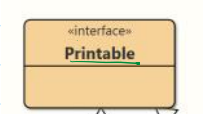
```
BlueJ: Terminal Window - AaditClass7
Options

This is outer interface method.

This msg() is nested interface method.

This printMsg() is nested interface method.
```

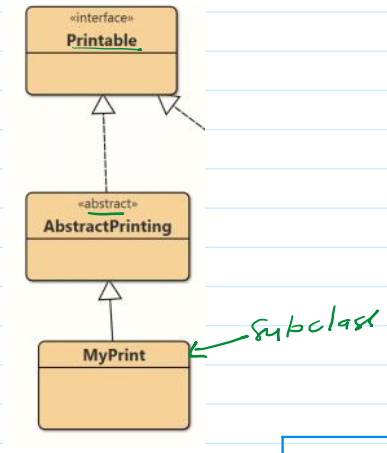
**Difference between Abstract class and Interface in Java**

In Java both abstract class and interface helps in achieving abstraction, but they are used in different ways. Abstract classes are for sharing common functionality among related classes, while interfaces define a set of methods that any class can implement.

The abstract class achieves partial abstraction (0 to 100%.) Whereas the interface achieves full abstraction (100%).

| **Abstract Class** | **Interface** |
|---|---|
| Abstract classes can have abstract and non-abstract methods. | An interface can have only abstract methods. Since Java 8 it can have default and static methods also. |
| Abstract class does not support multiple inheritance. | Interface supports multiple inheritance. |
| Abstract class can have final non final static and non-static variables. | An interface has only static and final variables. |
| Abstract class can provide the implementation of the interface. | An interface cannot provide the implementation of an abstract class. |
| The abstract keyword is used to declare an abstract class. | The interface keyword is used to declare an interface. |
| An abstract class can extend another Java class and implement multiple Java interfaces. | An interface can extend another Java interface only. |
| An abstract class can be extended using the Java keyword "extends". | An interface can be implemented using the Java keyword "implements". |
| A Java abstract class can have class members like private, protected, etc. | Members of Java interfaces are public by default. |






supports abstract methods, so, it is not compulsory to implement abstract method. The sub-class of this class must override the abstract method.

```
public interface Printable
{
    void print();
}

public abstract class AbstractPrinting implements Printable
{
}

public class MyPrint extends AbstractPrinting
{
    @Override
    public void print(){
        System.out.print("\nThis is AbstractPrinting.\n");
    }
}
```